

QB: Chapter 3.2 — Binary Search Trees & Advanced Trees

Unit 3 | 4×2 -mark | 8×10 -mark | Total: 12 questions Lectures: L36–L41
| BT Levels: BT1, BT2, BT5, BT6 | COs: CO2, CO3, CO4, CO5

Section A: 2-Mark Questions

Q1. [BT1 | CO2] Define the *BST property*. For a BST node N , state precisely what must be true of all values in its left subtree and its right subtree. Does this property apply recursively to every node?

Q2. [BT1 | CO3, CO4] State the two key differences between Bellman-Ford and Dijkstra's algorithm: (a) which type of edge weights each handles, and (b) the number of relaxation iterations each performs. What additional check does Bellman-Ford perform after the main iterations?

Q3. [BT2 | CO2] Describe the four AVL tree rotation cases: LL, RR, LR, and RL. For each, state: (a) what imbalance it fixes (which subtree is too heavy) and (b) whether a single or double rotation is performed.

Q4. [BT2 | CO2, CO4] Explain why B-trees are preferred over BSTs for database indexing. What specific property of a B-tree node minimises the number of disk I/O operations required to locate a key?

Section B: 10-Mark Questions

Q5. [BT5 | CO3, CO4] (*Experiment E9 — Non-overlapping Intervals, LeetCode #435*)

The Non-overlapping Intervals problem asks for the minimum number of intervals to remove so that no two remaining intervals overlap.

(a) Design the greedy algorithm. Prove — using an exchange argument — that sorting intervals by end time and greedily retaining non-overlapping intervals yields the minimum removals.

(b) Trace the algorithm on intervals = $[[1,2], [2,3], [3,4], [1,3]]$. Show the sorted order, the current retained interval after each comparison, and the count of intervals removed. Identify which intervals are excluded.

(c) State the time complexity ($O(n \log n)$ for sorting, $O(n)$ for the greedy scan). Explain how this greedy approach relates to the classical Activity Selection Problem. Discuss why a dynamic programming solution, while correct, is less efficient for this particular problem.

Q6. [BT5 | CO2, CO3]

A max-heap is to be built from array [3, 9, 2, 1, 4, 5, 8].

(a) Construct the max-heap using the bottom-up `build_max_heap` procedure (calling `max_heapify` from the last internal node up to the root). Draw the binary tree and show the array state after each `max_heapify` call.

(b) Perform three `extract_max` operations. After each extraction, show both the tree and array state. Trace the `sift_down` (heapify) operation following each swap of root with last element.

(c) Implement heap sort using this max-heap. Derive the time complexity of `build_max_heap` as $O(n)$ using the summation of `max_heapify` calls at each level. Show that overall heap sort is $O(n \log n)$. Compare heap sort with merge sort in terms of stability, space complexity, and practical performance.

Q7. [BT5 | CO3, CO5]

You are designing a data management module and must choose among: BST, AVL tree, B-tree, and hash table.

(a) Construct a comparison table listing average-case and worst-case time complexity for search, insert, and delete for each of the four structures.

(b) Analyse BST degradation when keys are inserted in sorted order. Draw the BST resulting from inserting {1, 2, 3, 4, 5} in this order. Show how height $h = O(n)$ leads to $O(n)$ operation costs, contrasting with a balanced tree of the same keys.

(c) For each of the following scenarios, recommend one data structure and justify with complexity arguments: (i) In-memory dictionary with 10^5 unique-key lookups per second. (ii) A database index on disk storing 10^9 records. (iii) A real-time task scheduler requiring $O(\log n)$ insert and $O(\log n)$ extract-min. (iv) A compiler symbol table requiring $O(1)$ average insert and lookup.

Q8. [BT6 | CO2, CO4]

Perform a complete BST construction followed by deletion. Start with an empty BST and insert: 15, 10, 20, 8, 12, 17, 25, 6 (in this order).

(a) Draw the BST after all insertions. Verify the BST property at the root and at nodes 10 and 20 by listing all keys in their respective subtrees.

(b) Delete node 10 using the standard three-case algorithm: (i) leaf, (ii) one child, (iii) two children. Identify which case applies, state the in-order successor (or predecessor) used if applicable, and draw the tree after deletion.

(c) Now delete the root node 15. Identify the case and the replacement node. Draw the final tree. Derive the time complexity of BST search, insert, and delete as $O(h)$ and demonstrate the $O(\log n)$ vs $O(n)$ contrast for a balanced tree vs a degenerate tree of 8 nodes.

Q9. [BT6 | CO2, CO4]

Starting with an empty AVL tree, insert keys in this order: 30, 20, 40, 10, 25, 5.

- (a) After each insertion, draw the tree, compute the balance factor of every node, and identify whether a rotation is required. Clearly name the imbalance case (LL, RR, LR, or RL) at the first imbalance encountered.
 - (b) Perform the rotation(s) required to restore balance. Draw the tree before the rotation, indicate the pivot node and rotation type, and draw the tree after the rotation.
 - (c) After all six insertions, draw the final AVL tree. Compute and record the balance factor of every node and verify all values are in $\{-1, 0, +1\}$. Compare the height of the final AVL tree with the height of a standard BST built from the same keys in the same insertion order.
-

Q10. [BT6 | CO2, CO4]

A B-tree of minimum degree $t = 2$ is built by inserting keys: 10, 20, 5, 6, 12, 30, 7, 17 (in this order).

- (a) State the minimum and maximum number of keys permitted per node for $t = 2$. Insert the first four keys and draw the resulting tree. Identify the first key insertion that triggers a node split and describe the split mechanism.
 - (b) Complete the full insertion sequence. Draw the tree after every split occurs. For each split, show: the overflowing node, the key promoted to the parent, and the two new child nodes formed.
 - (c) Explain why a B-tree always maintains perfect balance (all leaves at the same depth). Derive that the height h satisfies $h \leq \log_t((n+1)/2)$. Explain how this height bound guarantees $O(\log_t n)$ time for search, insert, and delete, and why this property is critical for database read performance.
-

Q11. [BT6 | CO2, CO4] (*Experiment E13 — Binary Tree Level Order Traversal, LeetCode #102*)

Level Order Traversal (LeetCode #102) returns a list of lists, where each inner list contains the node values at one level of the tree.

- (a) Design a BFS-based algorithm using a queue that performs level-order traversal with level grouping. Write the complete Python implementation. Explain how to determine when one level ends and the next begins (hint: use queue size at the start of each level's iteration).
 - (b) Trace the algorithm on the BST obtained by inserting keys $\{8, 3, 10, 1, 6, 14\}$ into an empty BST in this order. Draw the BST and then trace the level-order traversal: show the queue state at the beginning of each level and the level output after each level is processed.
 - (c) Modify the algorithm to produce *reverse* level-order traversal (deepest level first). State whether a stack, a second queue, or `list.reverse()` is the most appropriate tool and why. Derive the time complexity $O(n)$ and space complexity $O(w)$ (where w = maximum width of the tree) for standard level-order traversal.
-

Q12. [BT5 | CO3, CO4]

The Bellman-Ford algorithm computes single-source shortest paths and can detect negative-weight cycles.

(a) Describe the Bellman-Ford algorithm step by step. Explain: (i) why exactly $V - 1$ relaxation iterations are needed to guarantee shortest paths in a graph with V vertices, and (ii) how the V -th iteration is used to detect a negative-weight cycle.

(b) Trace Bellman-Ford on the directed graph: 5 nodes $\{S, A, B, C, D\}$, source = S , edges: $(S \rightarrow A, 6)$, $(S \rightarrow B, 7)$, $(A \rightarrow C, 5)$, $(B \rightarrow A, -3)$, $(B \rightarrow C, 8)$, $(B \rightarrow D, 9)$, $(C \rightarrow D, -4)$, $(D \rightarrow C, 7)$. Show the `dist[]` array after each of the 4 relaxation iterations.

(c) Provide a concrete small example graph (3 nodes, 3 edges) showing that Dijkstra's algorithm produces an incorrect shortest path when one edge has a negative weight. Derive and compare time complexities: Bellman-Ford $O(V \cdot E)$ vs Dijkstra with min-heap $O((V + E) \log V)$. State when each algorithm is the appropriate choice.